



University
of Glasgow

W3-03: Issues With Hadoop

COMPSCI4064 (H) and COMPSCI5088 (M)

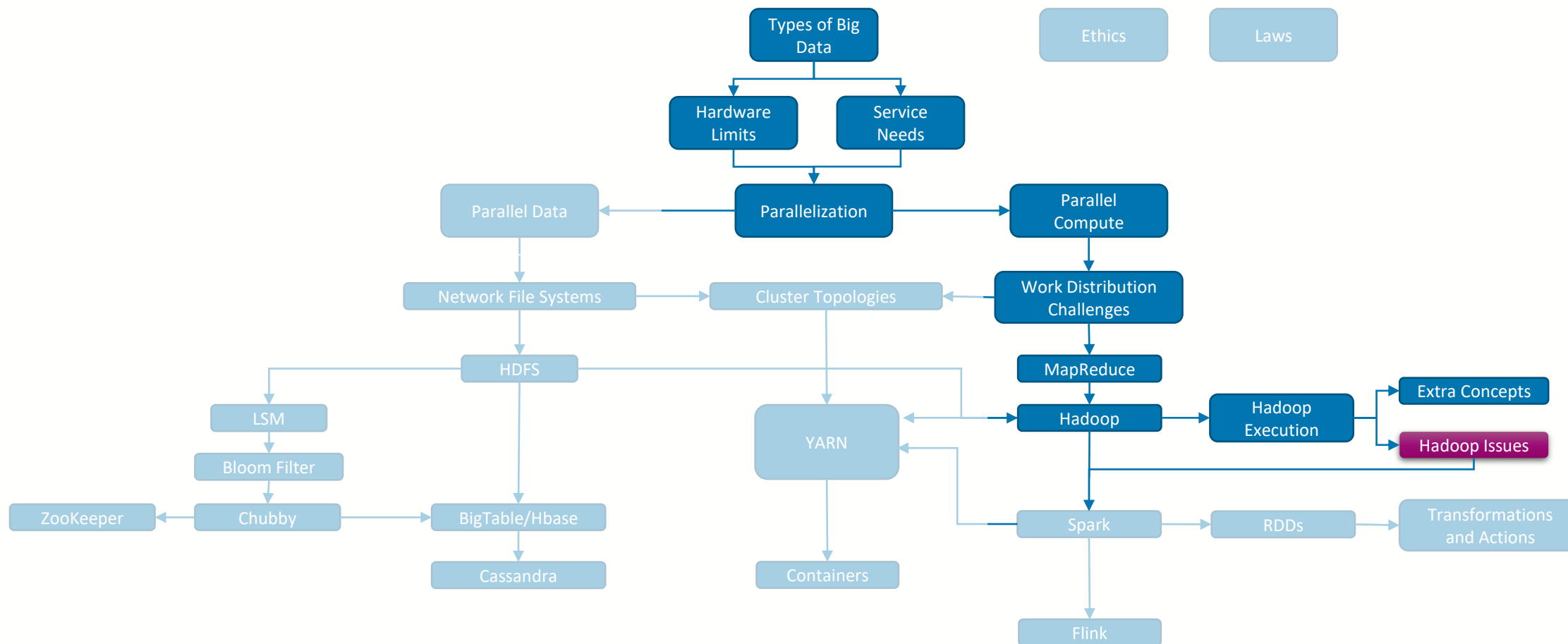
Dr. Richard McCreddie

WORLD
CHANGING
GLASGOW

A WORLD
TOP 100
UNIVERSITY



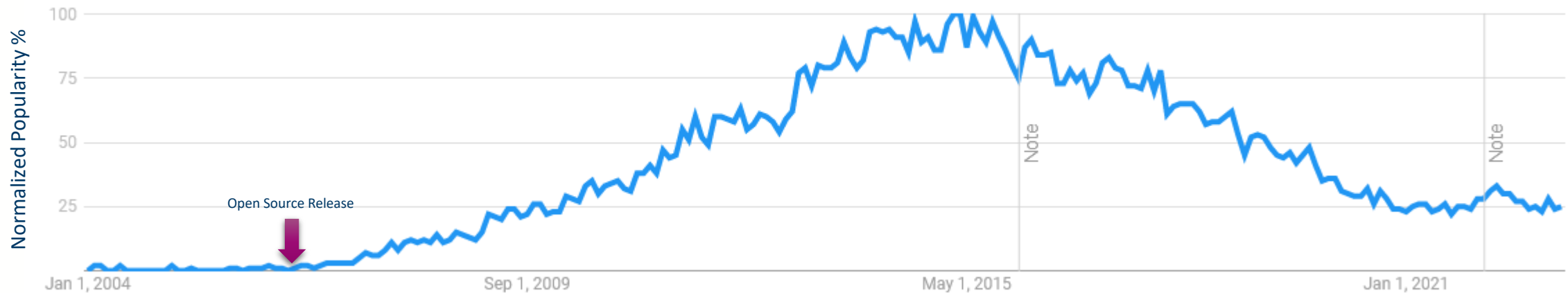
Where are we?





Declining Popularity

- When I introduced Hadoop I noted that it was declining in popularity...
 - Why is this?
- As it turns out, there are quite a few issues with Hadoop in practice, and since then better and more specialised tools have been produced, meaning that Hadoop is the right tool less often



Google Trends Chart for Hadoop



HDFS and Small Files

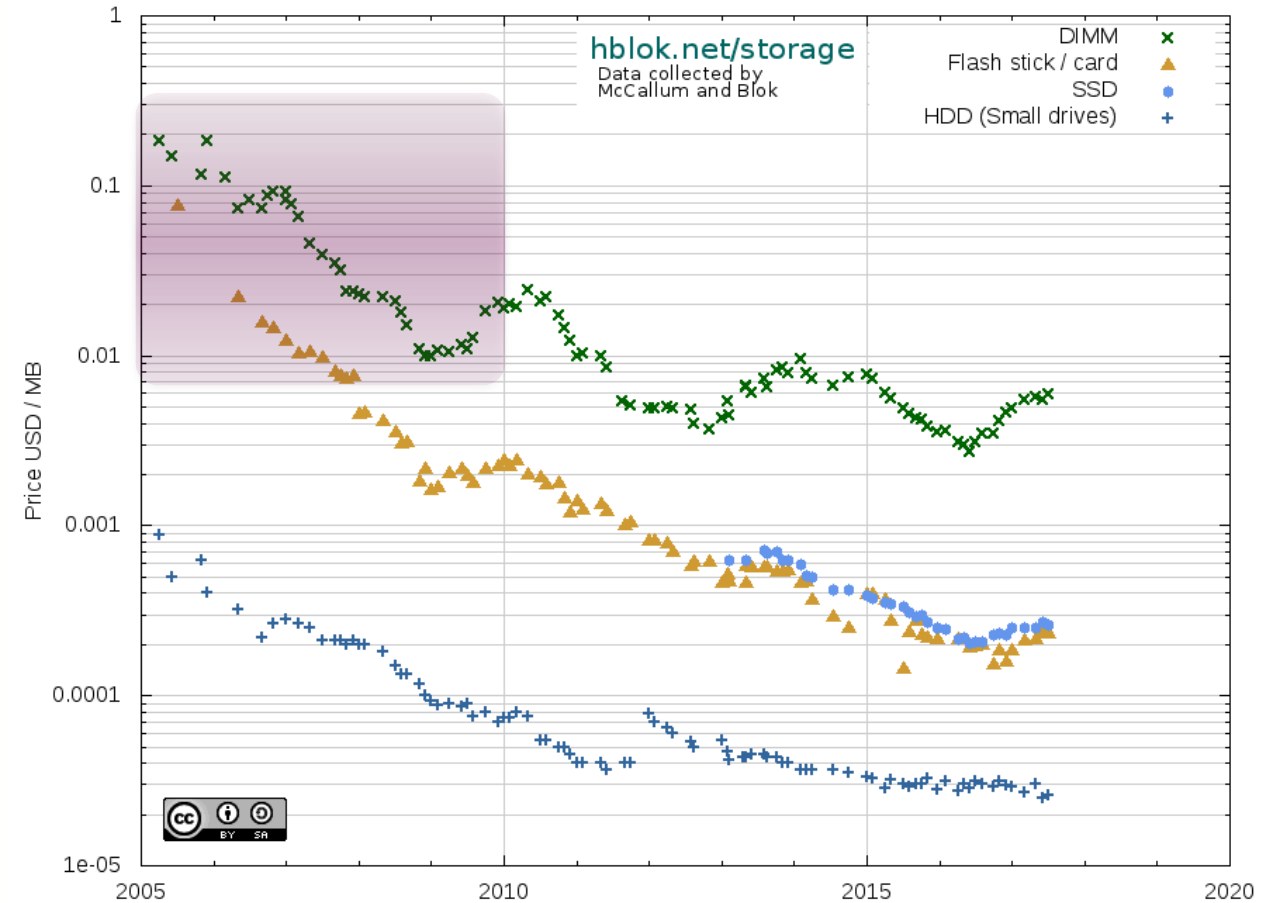
- We mentioned this in passing when discussing the input splits, but Hadoop is tied to the Hadoop distributed file system (HDFS)
 - The HDFS stores data in relatively large blocks of a fixed size (default 128MB)
 - If a file is smaller than this, each file needs a separate block entry in the Name Node index
- This means that if we have very large numbers of files, the Name Node index will become overloaded
- This issue can be overcome in various ways
 - Merging small files together
 - Using Hadoop Archives (HAR files), which enable a layered filesystem at the cost of slower index reads
 - Use Hbase to write binary data to a table rather than use files



Speed and Latency

- The second core reason that people are moving away from Hadoop is that it is slower than modern alternatives
 - Hadoop was designed back when Main Memory was small and expensive
 - This is one of the reasons why Hadoop writes to disk after every task
 - Hadoop has no caching
- Given the cheapness and size of Main Memory these days it makes more sense to do more work there

Historical Cost of Computer Memory and Storage





Cost of Chaining Maps and Reduces

- While it is true that many tasks can be represented by Map and Reduce operations, the issue is that most of these require **multiple Map and Reduce pairs**
 - In practice developers need to 'chain' MapReduce pairs in sequence to get the result they need
- Hadoop was **not** really **designed** to make this **easy or fast**
 - Data needs to be written to disk and then re-read between each link in the chain



What about those other Use-Cases?

- The other obvious reason that people are moving away from Hadoop is that there are an increasing number of use-cases that are not covered by Hadoop
 - Applications that need to **share state across instances of a task**
 - All **streaming** use-cases
 - Applications with significant **side-effects**



Ease of Use

- If we compare Hadoop to modern data science tools (e.g. Python's Pandas library), we see that Hadoop requires **all tasks to be hand-written**
 - Modern solutions use pre-built and optimised abstractions (e.g. lambda expressions) or specialist APIs to reduce the amount of code the programmer needs to write
 - The most popular language for writing these tasks is Java, which is also a wordy language



University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow